

PROD SECURITY CHECKLIST

23 MADDELIK STANDART

Uygulamayı Yayına Almadan Önce Claude / AI'a Yaptıracağın 23 Şey

Yapay zeka modelleri (Claude, ChatGPT, Cursor vb.) fonksiyonel kodu hızla üretir ancak güvenlik, gizlilik ve üretim seviyesi sıkılaştırma adımlarını kendiliğinden uygulamaz. Bu rehber; projenizi canlıya (production) almadan önce yapay zekaya yaptırmanız gereken 23 kritik güvenlik adımını, hazır prompt şablonlarını ve denetim maddelerini içerir.

DOKÜMAN ADI

App Security Pre-flight

KAPSAM

Frontend, Backend, DB & Cloud

HEDEF SEVİYE

Production Ready / OWASP

DURUM

Yayına Alım Öncesi Zorunlu

BÖLÜM 1: GİZLİ VERİLER, KİMLİK DOĞRULAMA VE OTURUM YÖNETİMİ

1 Anahtarları Koddan Çıkar (Secret & API Key Isolation)

Kritik Risk

Stripe, OpenAI, Supabase service-role gibi gizli anahtarlar asla istemciye giden kodlarda veya hard-coded olarak kaynak dosyalarda yer almamalıdır.

Claude Promptu: "Kod tabanımı tara. Hardcoded yazılmış tüm API key, secret token ve hassas bağlantı dizgilerini tespit et. Bunları `.env` dosyasına taşıyacak yapıyı kur ve istemciye asla expose edilmeyecek şekilde server-side ortam değişkenlerine bağla."

☐ Kontrol: Kaynak kodda hiçbir gizli anahtar kalmadı; `.env` oluşturuldu.

2 .env Dosyalarını Git Geçmişinden Sil (Purge Secrets from Git History)

Git Güvenliği

`.env` dosyasını `.gitignore`'a eklemek yetmez; geçmiş commit'lerde yer alan anahtarlar git geçmişinde açıkta kalır.

Claude Promptu: "Git commit geçmişimde yanlışlıkla commit edilmiş `.env` veya hassas dosyaları tespit etmek ve `git-filter-repo` / BFG Repo-Cleaner ile tüm commit geçmişinden kalıcı olarak temizlemek için adım adım terminal komutlarını yaz."

☐ Kontrol: Git log geçmişinde hassas veri kalmadı, ifşa olan tüm API anahtarları revoke edilip yenilendi.

3 İzin Kurallarını Yaz (Row Level Security & Storage Rules)

Veritabanı İzni

Firestore, Supabase, PocketBase veya PostgreSQL kullanırken varsayılan `allow read, write: if true`; kurallarını kaldırıp yetki bazlı RLS politikaları yazın.

Claude Promptu: "Veritabanı tablolarım için katı Row Level Security (RLS) politikaları yaz. Kullanıcılar yalnızca kendi `user_id`'lerine ait verileri okuyup güncelleyebilsin, genel tablolara sadece yetkili servisler yazabilsin."

☐ Kontrol: RLS aktif edildi; anonim veya yetkisiz kullanıcıların başka verilere erişimi test edilip engellendi.

4 Yetkiyi Sunucuda Tut (Server-Side Authorization Checks)

Bypass Önleme

Arayüzde bir butonu gizlemek güvenlik sağlamaz. API uç noktaları (endpoints) her istekte gelen token ve rolü sunucuda doğrulanmalıdır.

Claude Promptu: "Tüm backend endpoint'lerimi incele. Sadece frontend görünürlüğüne güvenen rotaları tespit et. Her API endpoint'ine JWT/oturum tabanlı, sunucu tarafında çalışan yetki ve rol doğrulama middleware'i ekle."

☐ Kontrol: cURL / Postman ile doğrudan API çağrıldığında yetkisiz istekler 401/403 dönüyor.

11 Şifreleri Hash'le (Strong Password Hashing)

Kriptografi

Kullanıcı şifreleri asla düz metin (plaintext) veya zayıf algoritmalarla (MD5/SHA1) saklanamaz. Güçlü tuzlama (salt) ve Argon2id / bcrypt kullanılmalıdır.

Claude Promptu: "Kullanıcı kayıt ve giriş fonksiyonlarımı güncelle. Şifreleri Argon2id veya bcrypt (work factor minimum 12) ile tuzlayarak hash'leyen ve güvenli şekilde doğrulayan fonksiyonları yaz."

☐ Kontrol: DB'de hiçbir düz şifre yok; modern hash algoritmaları ve yeterli iş faktörü devrede.

12 Çerezli Güvenli Yap (Secure, HttpOnly, SameSite Cookies)

Oturum Koruması

Oturum çerezleri JavaScript tarafından okunabilirse XSS saldırılarında çalınır. Çerez bayrakları sıkılaştırılmalıdır.

Claude Promptu: "Uygulamamdaki tüm oturum ve auth cookie tanımlamalarını HttpOnly: true, Secure: true ve SameSite: 'Strict' (veya 'Lax') bayraklarıyla yapılandır."

☐ Kontrol: document.cookie ile oturum anahtarına erişilemiyor; HTTPS dışı aktarım engelli.

BÖLÜM 2: AĞ, TRAFİK VE API GÜVENLİK YAPILANDIRMASI

5 Giriş Sınır Koy (Rate Limiting & Brute-Force Koruması)

DDoS / Brute-Force

Giriş (login), şifre sıfırlama, SMS/Email doğrulama ve pahalı LLM uç noktalarına IP/Kullanıcı bazlı istek limiti konmalıdır.

Claude Promptu: "Uygulamama (Redis / Upstash veya in-memory) tabanlı bir rate-limiter middleware'i ekle. Özellikle /api/login, /api/register ve maliyetli AI endpoint'lerine dakikada en fazla 5-10 istek hakkı tanı."

☐ Kontrol: Art arda yapılan denemelerde 429 Too Many Requests yanıtı alınıyor.

8 CORS'u Kendi Alanına Kilitle (Restrict Cross-Origin Sharing)

API İzolasyonu

Geliştirme aşamasında kullanılan Access-Control-Allow-Origin: * kuralı production'da sadece yetkili frontend alan adına kilitlenmelidir.

Claude Promptu: "CORS ayarlarımı düzenle. Wildcard (*) tanımlamasını kaldır. Yalnızca projemin canlı domaini (https://seninalanadi.com) ve belirlenen staging domainlerine izin veren sıkı bir CORS middleware'i yapılandır."

☐ Kontrol: Yetkisiz bir origin üzerinden API çağrısı yapıldığında tarayıcı isteği engelliyor.

9 Güvenlik Başlıklarını Ekle (HTTP Security Headers)

Tarayıcı Güvenliği

Clickjacking, MIME-sniffing ve inline-script saldırılarını engellemek için HTTP yanıt başlıkları (Helmet vb.) eklenmelidir.

Claude Promptu: "Sunucu yanıtlarıma `Strict-Transport-Security`, `Content-Security-Policy (CSP)`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff` ve `Referrer-Policy` başlıklarını ekleyen yapılandırmayı yaz."

☐ Kontrol: SecurityHeaders.com analizinde A+ veya A skoru alınıyor.

10 HTTPS Zorunlu Kıl (Enforce HTTPS & SSL/TLS)

Trafik Şifreleme

Tüm HTTP istekleri otomatik olarak HTTPS'e 301 yönlendirmesiyle yönlendirilmeli ve HSTS preload aktif edilmelidir.

Claude Promptu: "Uygulama sunucum ve reverse-proxy (Nginx / Cloudflare) için tüm HTTP isteklerini HTTPS'e kalıcı olarak yönlendiren ve HSTS başlığı ekleyen yapılandırma kodunu hazırla."

☐ Kontrol: `http://` protokolüyle gelen tüm bağlantılar anında `https://`'e yönlenecek.

17 Webhook İmzalarını Doğrula (Verify Webhook Signatures)

Ödeme / Entegrasyon

Stripe, LemonSqueezy, GitHub veya Iyzico webhook endpoint'leri gelen isteğin gerçekten sağlayıcıdan geldiğini doğrulamalıdır.

Claude Promptu: "Webhook endpoint'im için imza doğrulama fonksiyonu yaz. Gelen ham gövdeyi (raw body) kullanarak sağlayıcının HMAC-SHA256 imzasını gizli anahtar ile doğrula; geçersiz imzalı istekleri 400 ile reddet."

☐ Kontrol: İmzası olmayan veya sahte imzalı webhook istekleri işleme alınmıyor.

BÖLÜM 3: GİRDİ DOĞRULAMA, DOSYA GÜVENLİĞİ VE ENJEKSİYON ÖNLEME

6 Girdiyi Sunucuda Doğrula (Server-Side Input Validation)

Girdi Bütünlüğü

Frontend doğrulamaları kolayca atlatılabilir. Sunucuya gelen tüm veri tipleri, uzunlukları ve formatları şema doğrulayıcılarla (Zod, Joi vb.) kontrol edilmelidir.

Claude Promptu: "Tüm API endpoint'lerim için Zod / Joi şemaları oluştur. Gelen `req.body`, `req.query` ve `req.params` verilerini sıkı tip ve kural denetiminden geçiren doğrulama middleware'i ekle."

☐ Kontrol: Beklenmedik tip veya aşırı uzunluktaki girdiler sunucu tarafından anında reddediliyor.

7 Dosya Yüklemeyi Sınırla (File Upload Security & Limits)

Zararlı Dosya

Sınırsız dosya boyutu DoS yaratır; sadece dosya uzantısına güvenmek ise zararlı betiklerin (.exe, .php, .svg script) yüklenmesine yol açar.

Claude Promptu: "Dosya yükleme endpoint'ime katı kısıtlamalar ekle: Maksimum dosya boyutu sınırı (örn. 5MB), 'magic bytes' / buffer tabanlı gerçek MIME-type kontrolü ve yüklenen dosyaları rastgele UUID ile adlandırarak saklama mantığını yaz."

☐ Kontrol: Büyük dosyalar ve sahte uzantılı çalıştırılabilir dosyalar engelleniyor.

15 SQL Sorgusunu Parametrele (Parameterized Queries / No SQLi)

OWASP #1 SQLi

Dizgi birleştirme ("SELECT * FROM users WHERE id=" + id) doğrudan SQL Enjeksiyonuna neden olur. Her zaman parametrelili sorgu veya ORM kullanılmalıdır.

Claude Promptu: "Projemdeki tüm veritabanı sorgularını tara. String interpolation veya concatenation ile yazılmış ham sorguları tespit et ve tamamını prepared statements / parametrelili sorgulara dönüştür."

☐ Kontrol: Hiçbir ham string birleştirmeli SQL sorgusu kalmadı; SQLMap denemelerinde açık yok.

16 Çıktıyı XSS'e Karşı Kaçır (Cross-Site Scripting Sanitization)

XSS Koruması

Kullanıcı girdilerini DOM'a doğrudan `innerHTML` veya `dangerouslySetInnerHTML` ile basmak XSS açığı doğurur.

Claude Promptu: "Frontend kodlarımı tara. `innerHTML` veya güvensiz render yapılan yerleri bul. Girdileri HTML-escape eden veya `DOMPurify` ile sanitize eden güvenli render fonksiyonlarıyla değiştir."

☐ Kontrol: `<script>alert(1)</script>` içeren girdiler düz metin olarak güvenli gösteriliyor.

BÖLÜM 4: HATA YÖNETİMİ, LOGLAMA VE MALİYET KONTROLÜ**13 Hata Mesajlarını Kıs ve Sadeleştir (Error Masking in Prod)**

Bilgi İfşası

İstemciye veritabanı şeması, dosya yolları veya stack-trace içeren detaylı hatalar göndermek saldırganlara yol haritası sunar.

Claude Promptu: "Global error-handling middleware'i yaz. Production ortamında istemciye sadece genel hata mesajı (örn. 'Bir hata oluştu') ve benzersiz bir `errorId` dön; detaylı stack trace'i yalnızca güvenli sunucu loglarına yaz."

☐ Kontrol: 500 hatalarında kullanıcıya sistem iç detayları veya SQL hataları sızdırmıyor.

14 Loglardan Token ve Hassas Veriyi Temizle (Log Sanitization)

PII / KVKK / GDPR

Log kayıtlarına kazara düşen JWT token'lar, şifreler, kredi kartı veya kimlik bilgileri log sızıntılarında büyük felakete yol açar.

Claude Promptu: "Loglama altyapımı incele (Winston, Pino vb.). Loglara yazılan `req.body` ve `headers` içerisindeki 'password', 'token', 'authorization', 'creditCard' gibi hassas alanları otomatik maskeleyen (redact eden) bir filtre ekle."

☐ Kontrol: Canlı sunucu loglarında düz token veya şifre kaydı kalmadı.

22 Harcama ve Bütçe Uyarısı Kur (Cloud & LLM Billing Alerts)

Finansal Güvenlik

Bir bot saldırısı veya sonsuz döngü OpenAI / Claude API ya da Cloudflare faturalarını binlerce dolara çıkarabilir.

Claude Promptu: "Kullandığım bulut ve LLM sağlayıcıları (OpenAI, AWS, GCP, Vercel) için bütçe aşım alarmları (hard/soft limitler), webhook tabanlı Slack/Email harcama bildirimleri ve kullanıcı başı token tüketim kotası mekanizması tasarla."

☐ Kontrol: Aylık tavan limitler belirlendi; %50, %80 ve %100 eşiklerinde bildirim alarmları aktif.

BÖLÜM 5: YETKİLENDİRME, BAĞIMLILIKLAR VE VERİ YAŞAM DÖNGÜSÜ**18 Admin Uç Noktalarına Rol Koy (Role-Based Access Control)**

Yetki Matrisi

/admin veya kritik yönetim rotaları sadece oturum açmış kullanıcıyı değil, doğrulanmış role == 'ADMIN' yetkisini ve tercihen MFA'yı aramalıdır.

Claude Promptu: "Uygulamama Role-Based Access Control (RBAC) middleware'i kur. Admin rotalarını diğer kullanıcılardan izole et, yetkisiz erişimlerde 403 Forbidden dön ve tüm admin eylemlerini audit log'a kaydet."

☐ Kontrol: Standart kullanıcı hesabı ile admin API endpoint'lerine erişilemiyor.

19 Paketleri Güncelle ve Denetle (Dependency Vulnerability Audit)

Tedarik Zinciri

Kullanılan 3. parti kütüphanelerdeki bilinen güvenlik açıkları (CVE'ler) doğrudan sistemin ele geçirilmesine yol açabilir.

Claude Promptu: "package.json / requirements.txt dosyalarımı analiz et. npm audit veya pip-audit çıktılarını değerlendirerek kritik/yüksek riskli paketlerin güvenli sürümlerine yükseltme planını hazırla."

☐ Kontrol: npm audit veya Snyk taramasında 0 kritik/yüksek güvenlik açığı.

20 Yedeği Otomatiğe Bağla (Automated Backups & Recovery Plan)

İş Sürekliliği

Otomatik olarak alınmayan ve geri yükleme testi yapılmamış hiçbir yedek geçerli kabul edilemez.

Claude Promptu: "Veritabanım için günlük otomatik yedek alan, yedekleri farklı bir bulut depolama alanına (S3/R2) şifreli olarak yükleyen ve yedekten geri dönme (restore) adımlarını içeren bir Cron / Action scripti oluştur."

☐ Kontrol: Otomatik günlük snapshot aktif; test ortamında yedekten geri yükleme başarıyla denendi.

21 Hesabı Gerçekten Sil (GDPR / KVKK Compliant Data Deletion)

Yasal Uyumluluk

Kullanıcı hesabını sildiğinde yalnızca deleted: true bayrağı koyup kişisel verileri (email, telefon, dosyalar) açıkta bırakmak yasal cezalara yol açar.

Claude Promptu: "KVKK/GDPR uyumlu hesap silme mekanizması yaz. Kullanıcı silindiğinde kişisel verileri (PII) anonimleştiren, yüklediği dosyaları storage'dan temizleyen ve 3. parti servislerdeki (Stripe, Mailchimp) kayıtlarını kaldıran cascade silme fonksiyonunu kodla."

☐ Kontrol: Hesap silindiğinde tüm kişisel veriler ve storage nesneleri kalıcı olarak temizleniyor.

BÖLÜM 6: SALDIRGAN GÖZÜYLE SON TEST & DOĞRULAMA**23 Uygulamayı Saldırgan Gibi Dene (Red Teaming & Pentest Simulation)**

Penetrasyon Testi

Tüm geliştirmeler bittikten sonra uygulamayı kötü niyetli bir hacker perspektifiyle stres ve güvenlik testine tabi tutun.

Claude Promptu: "Kıdemli bir siber güvenlik penetrasyon testi uzmanı (Red Teamer) gibi davran. Projemin mimarisini ve endpoint listesini inceleyerek OWASP Top 10 zafiyetleri (IDOR, Broken Auth, Injection, SSRF, Mass Assignment vb.) odaklı 15 maddelik saldırı simülasyon testi listesi ve test payload'ları hazırla."

☐ Kontrol: Hazırlanan saldırı senaryoları test ortamında denendi; bulunan tüm açıklar kapatıldı.

YAYIN ÖNCESİ ONAY VE KONTROL MATRİSİ

No	Güvenlik Maddesi	Sorumlu / Alan	Durum	Onay
1	Anahtarları koddan çıkar (.env)	Backend / DevOps	Tamamlandı	[]
2	.env'i Git geçmişinden temizle	DevOps / Lead	Tamamlandı	[]
3	DB izin kurallarını ve RLS yaz	Database / Backend	Tamamlandı	[]
4	Yetki kontrollerini sunucuda yap	Backend Team	Tamamlandı	[]
5	Giriş ve API'lere Rate Limit koy	Backend / Infra	Tamamlandı	[]
6	Tüm girdileri sunucuda doğrula (Zod)	Backend Team	Tamamlandı	[]
7	Dosya yükleme boyutu & MIME sınırla	Backend / Storage	Tamamlandı	[]
8	CORS politikalarını domain'e kilitle	Backend / DevOps	Tamamlandı	[]
9	Güvenlik HTTP başlıklarını ekle (CSP/HSTS)	DevOps / Frontend	Tamamlandı	[]
10	HTTPS'i zorunlu kıl (SSL yönlendirme)	Infra / Cloudflare	Tamamlandı	[]
11	Şifreleri Argon2/bcrypt ile hash'le	Auth Service	Tamamlandı	[]
12	Oturum çerezlerini güvenli yap (HttpOnly)	Auth / Frontend	Tamamlandı	[]
13	Prod hata mesajlarını ve stack'i gizle	Backend Team	Tamamlandı	[]
14	Loglardan token/şifre/PII temizle	Observability	Tamamlandı	[]
15	Tüm SQL sorgularını parametrele	Backend / DB	Tamamlandı	[]
16	XSS'e karşı çıktıları temizle / escape et	Frontend Team	Tamamlandı	[]
17	Webhook HMAC imzalarını doğrula	Payment / Backend	Tamamlandı	[]
18	Admin uç noktalarına RBAC rol koy	Backend / Auth	Tamamlandı	[]
19	Paketleri denetle (npm audit / CVE)	Lead Developer	Tamamlandı	[]
20	Otomatik yedek ve geri yükleme testi	Database / Infra	Tamamlandı	[]
21	Hesap silmede PII ve dosyaları temizle	Backend / GDPR	Tamamlandı	[]
22	Bulut & LLM harcama alarmlarını kur	Finance / DevOps	Tamamlandı	[]
23	Saldırgan gibi pentest simülasyonu yap	Security / QA	Tamamlandı	[]

Geliştirme Lideri

İsim:

Tarih:

İmza:

Güvenlik / QA Sorumlusu

İsim:

Tarih:

İmza:

Proje Yöneticisi / PO

İsim:

Tarih:

İmza: